

## **Task 10**

### **What is multi-threading programming ??**

Multi-threading is a programming technique where a single process runs multiple threads concurrently to perform different tasks simultaneously.

### **What is thread??**

A thread is the smallest unit of execution within a process. Think of it as a separate path of execution that can run code independently.

Multi-threading allows your program to do multiple things at the same time, making it faster and more responsive. Modern C# uses `async/await` and `Task` which are easier than managing threads manually.

# Report

## 1.What is State Management ?

State management is the process of persisting data across multiple requests in a stateless protocol like HTTP.

| Type              | Description                    | Example                               |
|-------------------|--------------------------------|---------------------------------------|
| Client-Side State | Stored on user's browser       | Cookies, LocalStorage, SessionStorage |
| Server-Side State | Stored on server               | Session, Cache, Database              |
| Distributed State | Shared across multiple servers | Redis, SQL Server State               |

## 2.What is Caching?

Caching is a technique to store frequently accessed data in fast, temporary storage to reduce database load and improve performance.

## 3.What is Redis?

Redis (Remote Dictionary Server) is an in-memory data structure store used as:

Cache

Database

Message Broker

## 4.Installing Redis??

### 1.Local Installation (Windows)

### 2.Docker (Recommended)

### 3.Redis Cloud (Free Tier)

## 5.Adding Redis to [ASP.NET](#) Core Project

1.Install NuGet Package

2.Configure in [Program.cs](#)

3.Add Connection String

## **6.Using Redis Cache Code**

- 1.IDistributedCache (Simple)
- 2.Advanced with Connection Multiplexer

## **7.Cache Strategies**

- 1.Cache-Aside(Lazy Loading)
- 2.Write-Through
- 3.Write-Behind (Write-Back)
- 4.Cache Eviction Policies
- 5.Redis Data Structures & Use Cases

Redis is the industry standard for distributed caching. It provides sub-millisecond latency, rich data structures, and built-in expiration. In ASP.NET Core, use `IDistributedCache` for simple scenarios or `StackExchange.Redis` for advanced features.